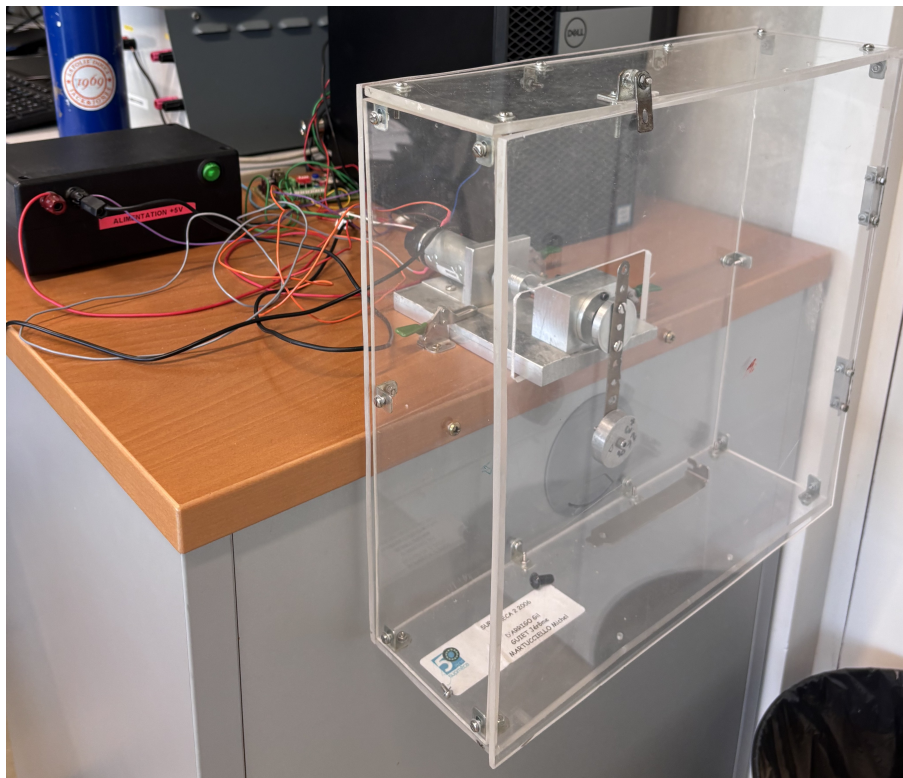




Pendule Rigide

CLAVERIE Timm
DUBUC Julien

3A Ecole d'Ingénieur
Parcours SYSMER
2026-2027



Résumé

Ce rapport illustre les étapes de modélisation et de contrôle d'un système de pendule rigide. Les TP s'appuient sur MATLAB et Simulink pour concevoir et tester les lois de commande. L'étude est structurée autour de 3 approches : MIL (Model In the Loop), SIL (Software In the Loop) et HIL (Hardware In the Loop). Les méthodes de contrôle concernent l'utilisation de correcteur PID, de retour tachymétrique et de commande linéarisante.

Mots clés : Pendule rigide, MIL, SIL, HIL, MATLAB, Simulink, PID, Retour tachymétrique, Commande linéarisante.

Table des matières

1	Introduction	2
2	MIL (Model In the Loop)	3
2.1	Description du système	3
2.2	Modélisation du moteur à courant continu	3
3	SIL (Software In the Loop)	4
3.1	Modélisation du moteur à vide	4
3.2	Représentation d'états et Analyse Linéaire	5
3.3	Conception d'un correcteur (Asservissement vitesse)	6
3.4	Modélisation avec balourd et Commande Linéarisante	6
4	HIL (Hardware In the Loop)	8
4.1	Identification des paramètres moteur (sans charge)	8
4.2	Identification des paramètres du pendule (essai quasi-statique)	8
4.3	Identification du frottement sec	9
4.4	Commande en boucle fermée	10
4.4.1	PID en position (simulation)	10
4.4.2	PID sur système réel	11
4.4.3	Retour tachymétrique (simulation)	12
4.4.4	Retour tachymétrique sur système réel	13
4.5	Test supplémentaire	14
5	Conclusion	15
6	Annexe	16
6.1	Tableau des paramètres	16
6.2	Codes Matlab	16

1 Introduction

Ce TP a pour objectif de piloter un pendule rigide monté sur un arbre motorisé horizontal.

L'étude débute par une phase MIL (Model In the Loop), qui consiste à modéliser la partie opérative du système à partir de ses équations fondamentales.

Puis, vient la partie SIL (Software In the Loop) dédiée à l'élaboration d'une loi de commande linéarisante simulée avec Simulink, en s'appuyant uniquement sur les données fournies par le constructeur.

Enfin, la partie HIL (Hardware In the Loop) permettra de procéder à l'identification des paramètres réels du système physique, incluant les frottements, l'inertie et le balourd. Cette implémentation matérielle sert finalement à réaliser la synthèse et la comparaison des performances de deux correcteurs (un PID de position et un retour tachymétrique) sur le système réel.

2 MIL (Model In the Loop)

Nous avons commencé par modéliser la partie opérative du système à partir des équations fondamentales.

2.1 Description du système

Le système est constitué d'un moteur à courant continu AMAX32G (voir Figure 2.2) et d'un codeur incrémental de 500 impulsions par tour.

L'arbre moteur entraîne directement le pendule rigide (voir Figure 2.1)). Pour l'étude MIL/SIL, la longueur du pendule est $L = 10cm$ et d'inertie négligeable par rapport à la masse $M = 30g$. Pour l'étude HIL, la longueur est $L = 8cm$ et d'inertie négligeable par rapport à la masse $M = 29g$ visée sur le pendule (à un trou du bout).

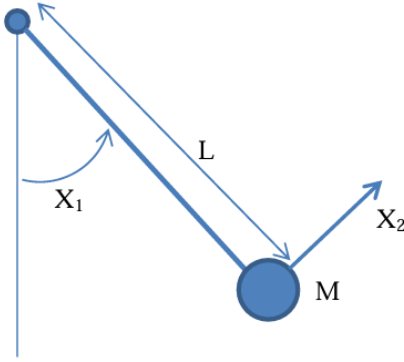


FIGURE 2.1 – Schéma du système

1	Tension d'alimentation (Ua)	V	12
2	Vitesse au courant In	tr/mn	2754
3	Couple au courant In	mNm	38.2
4	Courant max permanent (In)	mA	1580
5	Vitesse à vide à Ua à +/- 10%	tr/mn	4630
6	Courant à vide à +/- 50%	mA	85.9
7	Couple de démarrage à Ua	mNm	97
8	Courant de démarrage à Ua	mA	4010
9	Constante de couple	mNm/A	24.1
10	Constante de vitesse	tr/mn/V	396
11	Pente vitesse/couple	tr/mn/mNm	49.1
12	Vitesse limite	tr/mn	6000
13	Puissance utile max. à Ua	W	12
14	Rendement maximum	%	72.6
15	Constante de temps électromécanique	ms	21.3
16	Inertie	gcm²	41.4
17	Résistance aux bornes	Ohm	2.99
18	Inductivité	mH	0.42
19	Résistance thermique Boîtier/Ambiant	K/W	7.5
20	Résistance thermique Rotor/Boîtier	K/W	2.1

FIGURE 2.2 – Spécifications AMAX32G

2.2 Modélisation du moteur à courant continu

Les équations électriques et mécaniques gouvernent le moteur avec la masse.

L'équation électrique est :

$$U_a = L \frac{di}{dt} + Ri + K_e \Omega \quad (2.1)$$

L'équation fondamentale de la dynamique sur l'arbre moteur avec le balourd est :

$$J \frac{d\Omega}{dt} = K_c i - b\Omega - C_r - F_s \quad (2.2)$$

Le couple résistant C_r est dominé par l'effet gravitationnel du pendule qui est non linéaire selon θ :

$$C_{balourd} = mgl \sin(\theta) \quad (2.3)$$

Ce modèle physique sert de base pour la représentation d'état sous MATLAB et les simulations.

3 SIL (Software In the Loop)

Après le MIL, nous sommes passés au SIL sur MatLab/Simulink pour pouvoir simuler le moteur.

3.1 Modélisation du moteur à vide

Nous avons d'abord implémenté le schéma bloc traduisant les équations électriques et mécaniques du moteur seul, avec $C_s = 0$ et $F_s = 0$ (voir Figure 3.1).

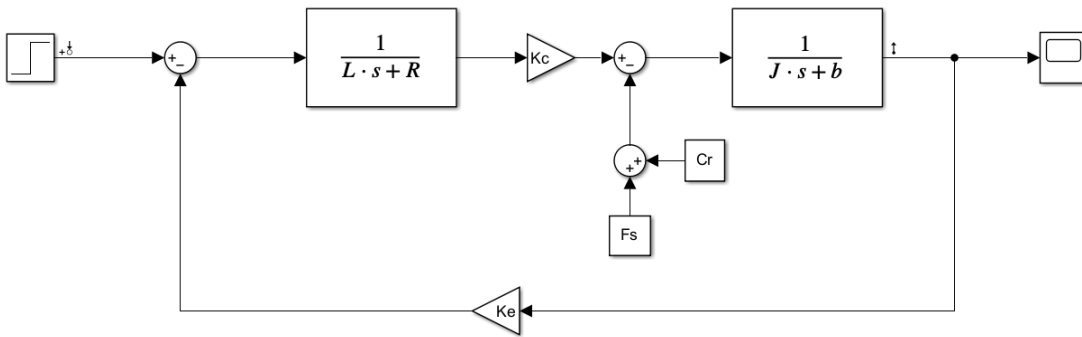


FIGURE 3.1 – Schéma bloc Simulink du moteur



FIGURE 3.2 – Réponse de la vitesse à un échelon de 12V

La vitesse finale du moteur simulé est de $487.2 \text{ rad/s} = 4652.4 \text{ tr/min}$ ce qui correspond quasiment à la vitesse à vide de 4630 tr/min annoncée par le constructeur du moteur. Le temps de réponse est atteint à la vitesse $0.95 * 487.2 = 463 \text{ rad/s}$, donc $t_{5\%} = 0.063 \text{ s}$. On est là aussi proche du constructeur avec $t_{5\%} = 3 * \tau = 0.0639 \text{ s}$. Pour obtenir le coefficient de frottement visqueux b , nous avons appliqué le théorème de la valeur finale : $b = \frac{K_c I_{vide}}{\Omega_{\infty}} = 4.27 * 10^{-6} \text{ N.m/(rad/s)}$.

3.2 Représentation d'états et Analyse Linéaire

On peut représenter le système sous forme de matrices :

$$\dot{X} = AX + BU \quad (3.1)$$

$$Y = CX + DU \quad (3.2)$$

avec $X = [i \ \Omega]^T$, $U = [U_a \ 0]^T$ et $Y = [\Omega]$. La représentation du système est :

$$\begin{bmatrix} \frac{di}{dt} \\ \frac{d\Omega}{dt} \end{bmatrix} = \begin{bmatrix} -\frac{R}{L} & -\frac{K_e}{L} \\ \frac{K_c}{J} & -\frac{b}{J} \end{bmatrix} \begin{bmatrix} i \\ \Omega \end{bmatrix} + \begin{bmatrix} \frac{1}{L} & 0 \\ 0 & -\frac{1}{J} \end{bmatrix} \begin{bmatrix} U_a \\ 0 \end{bmatrix} \quad (3.3)$$

$$Y = \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} i \\ \Omega \end{bmatrix} \quad (3.4)$$

Nous avons ensuite placé des points de linéarisation sur le schéma Simulink et avec l'outil Model Linearizer, on a extrait les pôles et zéros du système (voir Figure 3.3) et sa réponse à un échelon (voir Figure 3.4). Cette analyse linéaire nous permet d'obtenir la valeur du gain

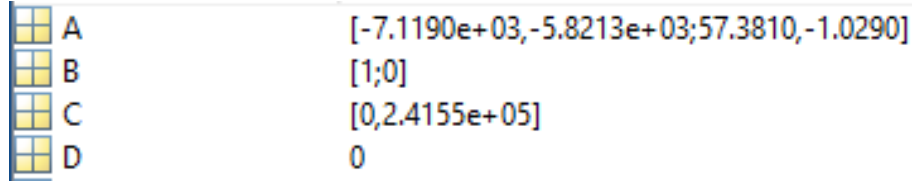


FIGURE 3.3 – Matrices du système

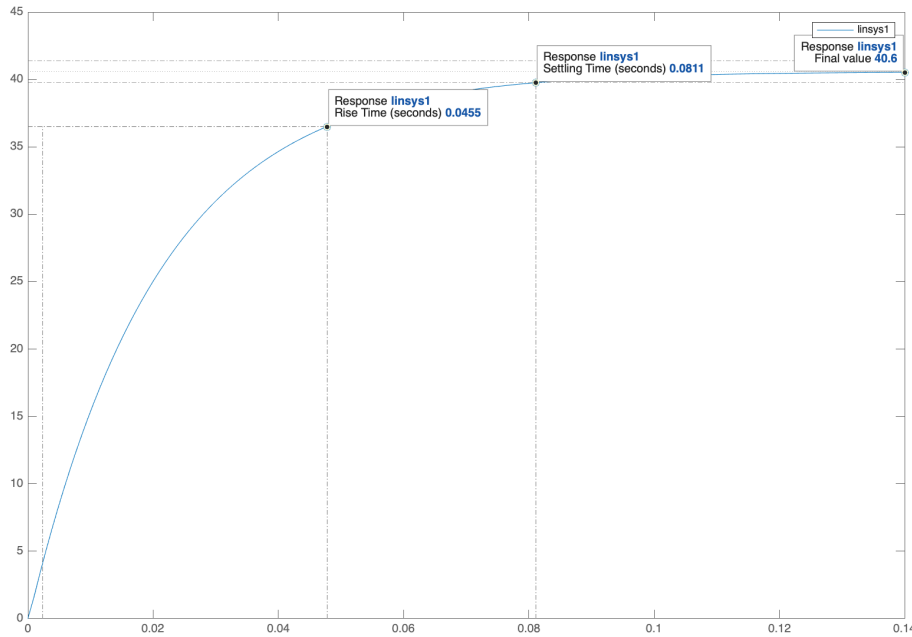


FIGURE 3.4 – Réponse à un step

statique $dcgain(linsys1) = 40.57$, le temps de montée *Rise Time* = 0.045s, le temps de réponse *Settling Time* = 0.081s et la valeur finale *Final value* = 40.6. Les pôles sont environ -48 rad/s et -7070 rad/s , sans zéro.

3.3 Conception d'un correcteur (Asservissement vitesse)

Pour asservir la vitesse du moteur, nous avons ajouté une boucle, intégré un bloc PID (voir Figure 3.5) et appliqué une consigne en vitesse (échelon) pour observer la vitesse en sortie (voir Figure 3.6).

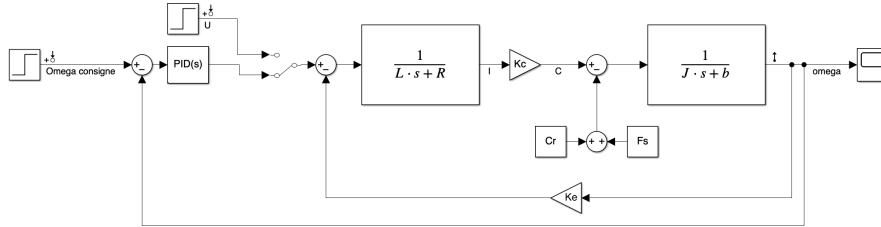


FIGURE 3.5 – Schéma Asservissement en vitesse

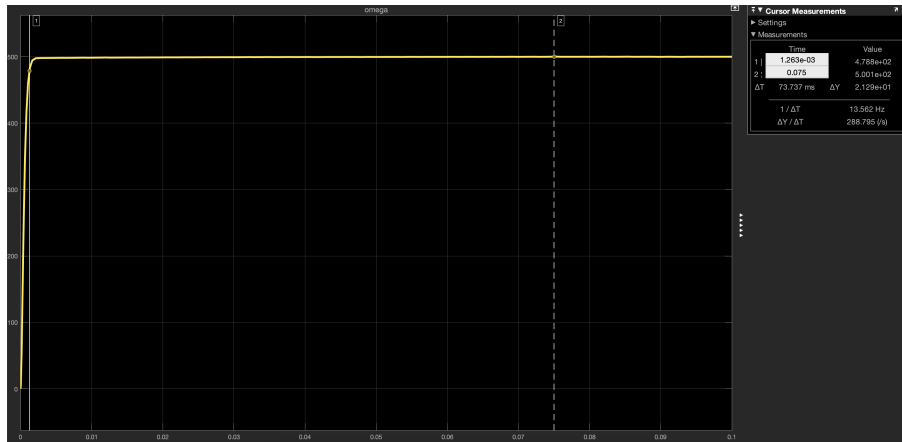


FIGURE 3.6 – Réponse de la vitesse avec le PID

Nous avons réglé le PID ($K_p = 1$ et $K_i = 40$) en vérifiant directement la réponse temporelle sur le scope. Nous avons obtenu de bonnes performances : la valeur finale de 500 rad/s est égale à la consigne avec une erreur statique nulle et un temps de réponse à 5% d'environ 1,2 ms sans dépassement qui est théorique.

3.4 Modélisation avec balourd et Commande Linéarisante

Nous avons ensuite considéré la charge du pendule, ce qui rend le système non linéaire, $C = MgL \sin(\theta)$ et dégrade les performances.

Pour réduire cette dégradation, nous avons implémenté une commande linéarisante par bouclage (voir Figure 3.7) qui tente d'annuler en temps réel la gravité pour imposer la commande :

$$U = J \cdot V + MgL \sin(\theta) \quad (3.5)$$

Grâce à ce bouclage, la non-linéarité est annulée et le système se comporte comme une double intégration linéaire ($\dot{\theta} = V$). V est asservie par un correcteur PD avec $\alpha_0 = 25$ et $\alpha_1 = 10$ comme

gains.

$$V = 25(\theta_{ref} - \theta) + 10(\dot{\theta}_{ref} - \dot{\theta}) + \ddot{\theta}_{ref} \quad (3.6)$$

Ces gains garantissent un comportement stable répondant à l'équation caractéristique $(s+5)^2 = 0$. La simulation montre que le pendule suit sa trajectoire.

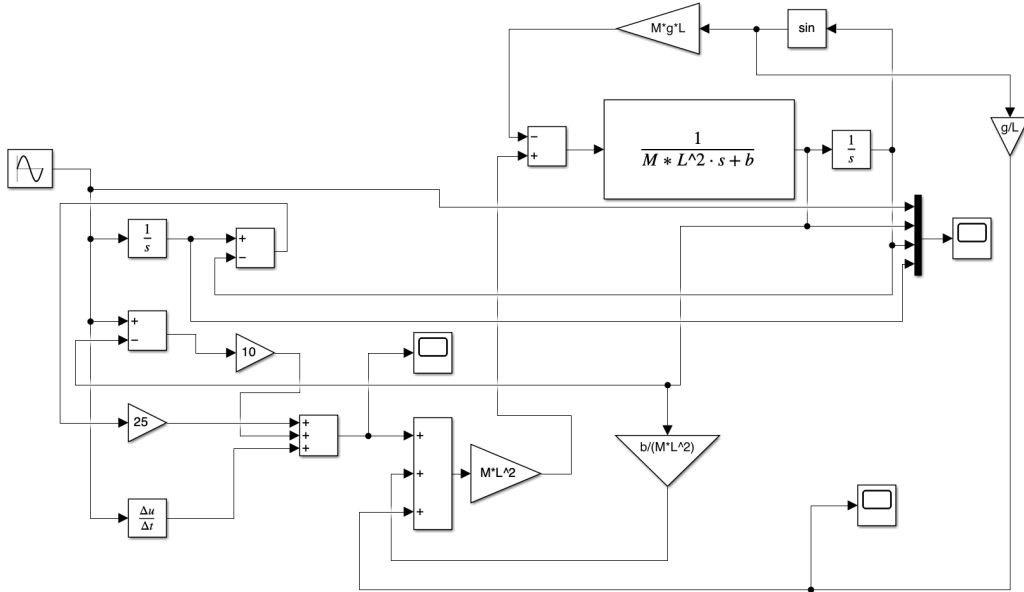


FIGURE 3.7 – Schéma Simulink de la commande linéarisante par bouclage

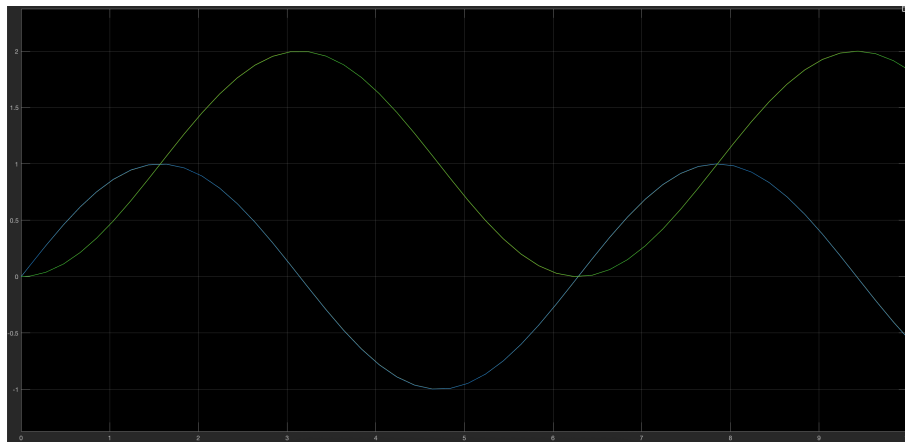


FIGURE 3.8 – Suivi de trajectoire avec la commande linéarisante

Les résultats montrent que la non linéarité gravitationnelle est compensée (voir Figure 3.8). Le scope reçoit 4 signaux mais uniquement 2 courbes sont visibles : la position réelle se superpose parfaitement à sa consigne (en vert) et de même pour la vitesse avec la sienne (en bleu).

Ce bouclage reste sensible aux perturbations physiques, comme à un jeu dans le codeur ou l'ajout d'un fort frottement visqueux qui crée une erreur de suivi. Il est donc nécessaire de tester nos modèles physiquement.

4 HIL (Hardware In the Loop)

Nous avons deux modèles un théorique et un pratique. Nous affinons les données du modèle théorique pour se rapprocher de celui réel. Dans la première partie du TP, nous avons modélisé l'actionneur sans charge, identifié un ensemble de paramètres, puis bouclé notre système avec une méthode d'asservissement par PID et par Tachymétrie.

4.1 Identification des paramètres moteur (sans charge)

Moteur seul (pendule démonté), avec un essai en échelon de tension, et l'hypothèse de réponse assimilable à un système du 1er ordre. On calcule le frottement visqueux et l'inertie équivalente.

Frottement visqueux b :

Il est identifié par le théorème de la valeur finale sur la réponse en vitesse à l'échelon. Vitesse en régime établi relevée à l'essai.

$$\omega_{\infty} = \frac{K_c U_0}{Rb + K_c K_e}$$

$$b = \frac{K_c U_0 - K_c K_e \omega_{\infty}}{R \omega_{\infty}} = 7.2 * 10^{-5} Nm/rad/s.$$

Inertie équivalente J_{eq} :

Elle est identifiée à partir du temps de réponse de la réponse du 1er ordre.

$$J_{eq} = \frac{\tau K_c U}{R \omega_{\infty}} = 6.87 * 10^{-5}.$$

Ces valeurs sont bien au-dessus des spécifications du moteur seul, car on prend en compte l'inertie et les frottements réels.

4.2 Identification des paramètres du pendule (essai quasi-statique)

Avec une rampe de tension lente (conditions quasi-statiques, vitesse angulaire ≈ 0 à chaque instant), le couple moteur équilibre exactement le couple de gravité du balourd. Le franchissement de l'horizontale donne directement le couple moteur nécessaire pour vaincre le balourd, d'où sa masse/inertie équivalente ($C_r = mgl$).

Le premier tour est identifié sur la zone de forte pente de la courbe angle/temps de la rampe car la masse tombe (voir Figure 4.1).

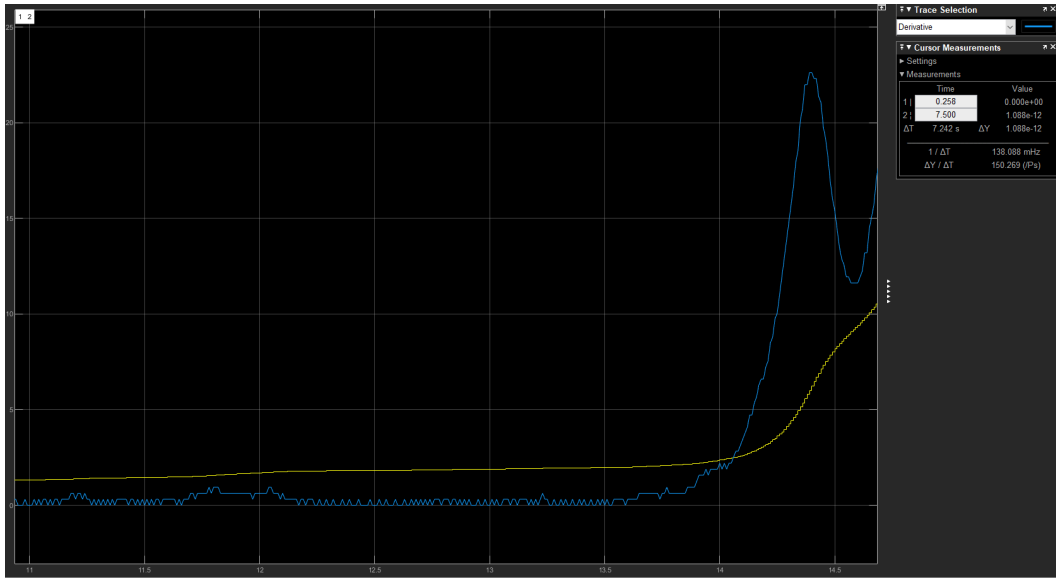


FIGURE 4.1 – Rampe de tension visualisation du passage d'angle horizontal

La longueur du pendule est de $0.08m$ et sa masse de $0.029kg$. On a une tension de basculement à $3.3V$ aux bornes du moteur, ce qui donne $C_r = \frac{u \cdot K_c}{R} = 0.0266 N.m$. Ce résultat est cohérent avec la valeur théorique $mgl = 0,0228 N.m$. On a $J_{eq} = ml^2 + J = 2.54 \cdot 10^{-4}$.

On implémente ensuite une commande pour maintenir le pendule dans n'importe quelle position initiale (compensation feedforward de la gravité).

Cependant, cela fonctionne uniquement en statique. En dynamique, la moindre perturbation entraîne un tour complet. La compensation gère l'équilibre statique, pas le mouvement. On observera aussi un écart de maintien : la tension réelle de la carte d'amplification est mesurée à 2,1 au lieu de 2,0 nominal, ce décalage de calibration expliquant l'imperfection du maintien en position. La qualité de la carte électrique est donc un élément limitant notre système.

4.3 Identification du frottement sec

Lors d'un essai en échelon de tension en BO (pendule monté), on a constaté l'écart entre la simulation sans frottement sec et la mesure réelle.

	Time	Value
1	2.500	9.510e+03
2	7.500	9.510e+03

FIGURE 4.2 – Simulation sans frottement

	Time	Value
1	2.500	-6.732e+03
2	7.500	-6.912e+03

FIGURE 4.3 – Mesure réelle sur le moteur

La valeur de vitesse était négative 4.3, elle a ensuite été corrigée. La différence entre les deux valeurs 4.2 finales est de $\frac{9510-6732}{9510} = 29,2\%$.

Pour corriger cet énorme écart, on a ajouté un bloc Dead Zone au niveau du couple moteur dans le modèle que l'on a réglé empiriquement pour rapprocher les deux courbes. Cependant même avec

une valeur de $[-0.5; 0.5]V$ l'écart restait conséquent, il faut donc implémenter un asservissement en fermant la boucle.

4.4 Commande en boucle fermée

Pour améliorer le contrôle de pose, nous abordons une méthode par correction PID et tachymétrie.

4.4.1 PID en position (simulation)

On a implémenté un correcteur PID ($K_p = 1.5$; $K_i = 0$ et $K_d = 0.5$) (voir Figure 4.4). On rentre une consigne verticale = π rad. On vient régler le gain proportionnel seul en premier et ensuite la composante dérivée.

Il n'y a pas de réglage automatique (Tune/Ziegler-Nichols) : la linéarisation du système autour du point de fonctionnement annulerait les couples résistants (gravité, frottements), et serait non représentatif du comportement réel du pendule (équivalent d'avoir un pendule tourné de 90 deg).

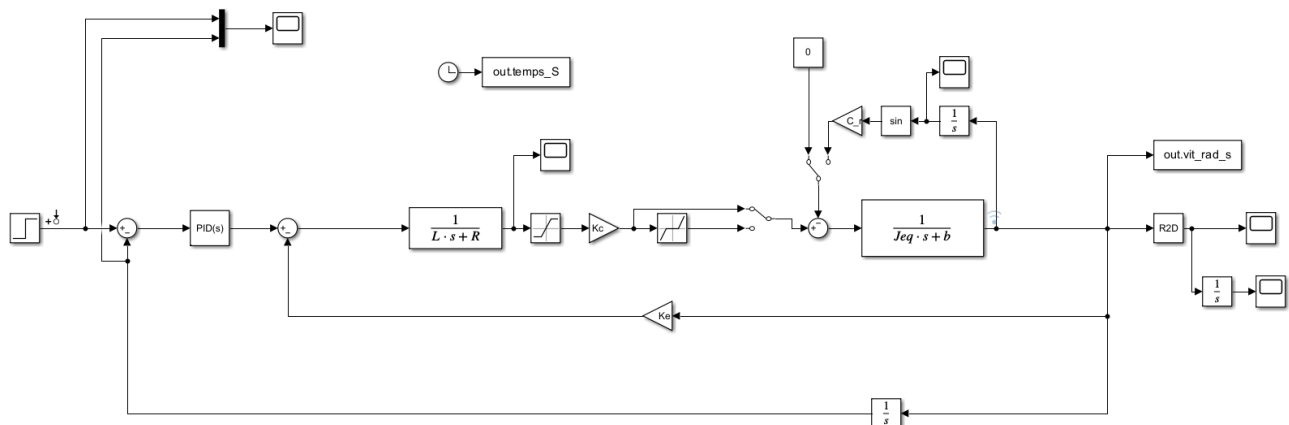


FIGURE 4.4 – Schema simulink de l’asservissement en position par PID en simulation



FIGURE 4.5 – Asservissement en position par PID en simulation

La réponse est stabilisée sans oscillation en $1.8s$, voir 4.5. La consigne est bien appliquée.

4.4.2 PID sur système réel

Nous avons implémenté le PID sur le système réel avec comme entrée, l'Encoder Input, et comme sortie, l'Encoder Output (voir Figure 4.6). Sur la figure 4.7, on voit que la consigne de 180° n'est pas atteinte. Le moteur donne des à-coups mais stagne à un plateau bas ($\approx 20-60^\circ$) pendant les 8 premières secondes car le couple de gravité devient maximal vers l'horizontal. Puis à $t = 8s$, on aide le moteur en déplaçant la masse à la verticale. Il continue à donner des à-coups pour se stabiliser. Cependant en faisant cela il devient instable et finit par tomber.

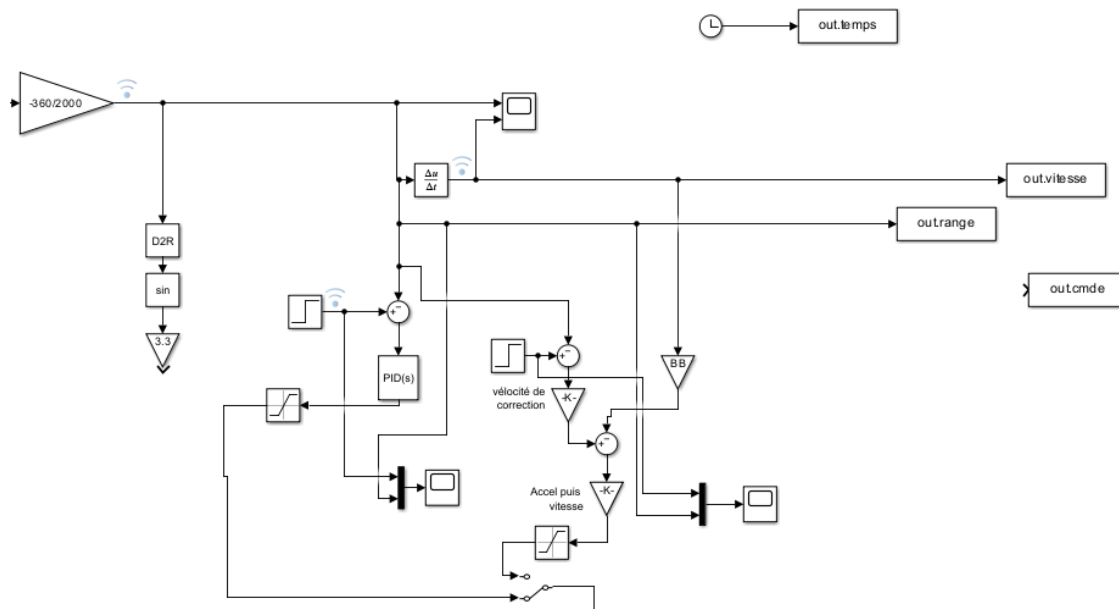


FIGURE 4.6 – Schema simulink de l'asservissement en position par PID en test physique

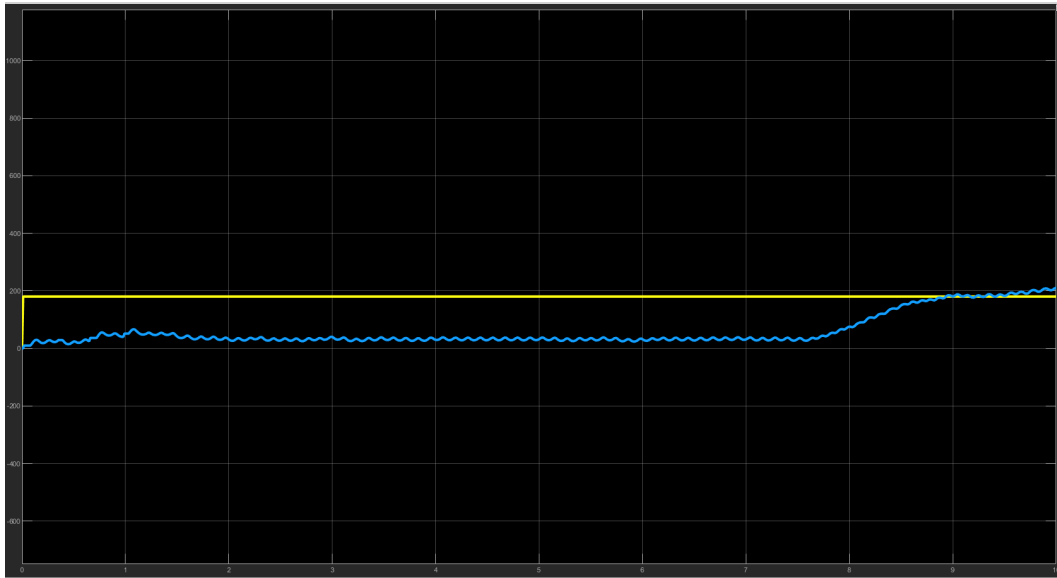


FIGURE 4.7 – Asservissement en position par PID en test physique

La cause est que le couple moteur seul est insuffisant pour sortir le pendule de sa position basse car le couple résistant est maximal. Une aide manuelle est donc nécessaire au démarrage.

Ainsi la stabilisation est imparfaite en régime établi, un correcteur linéaire appliqué à un système non-linéaire (couple de gravité) ne compense pas correctement toute la plage angulaire. Nous devons donc remplacer le PID par un correcteur avec retour tachymétrique.

4.4.3 Retour tachymétrique (simulation)

Nous avons une structure à 2 boucles : une boucle externe de position (gain K1) et une boucle interne de vitesse (gain K2, retour tachymétrique de gain B) (voir Figure 4.8).

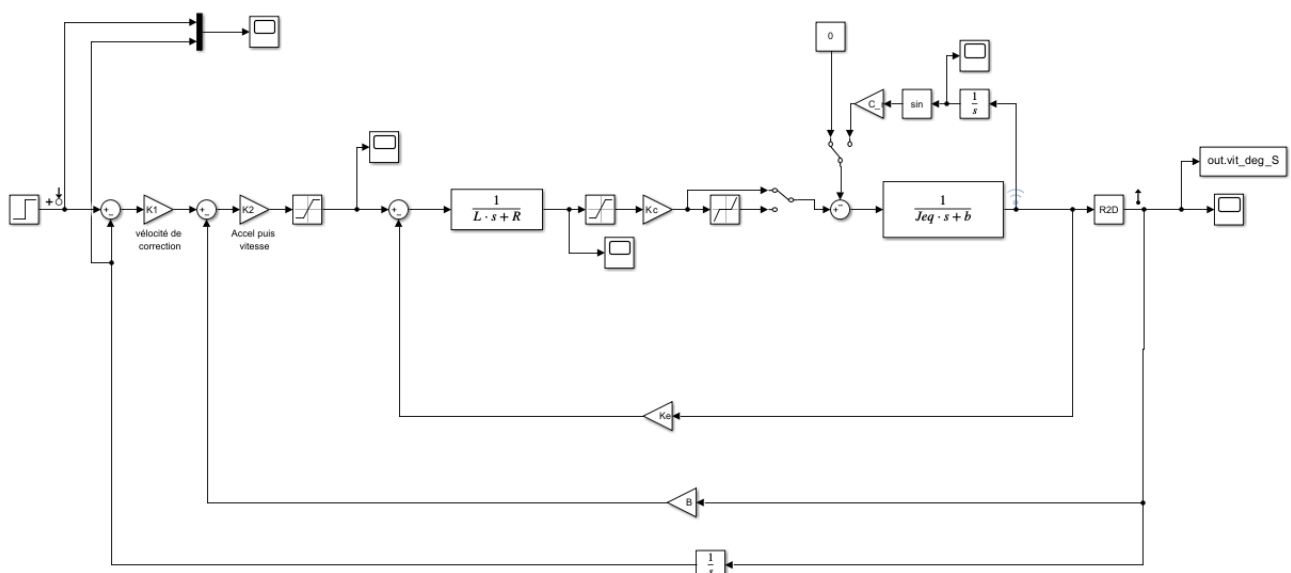


FIGURE 4.8 – Schema simulink de l'asservissement en position par tachymétrie en simulation

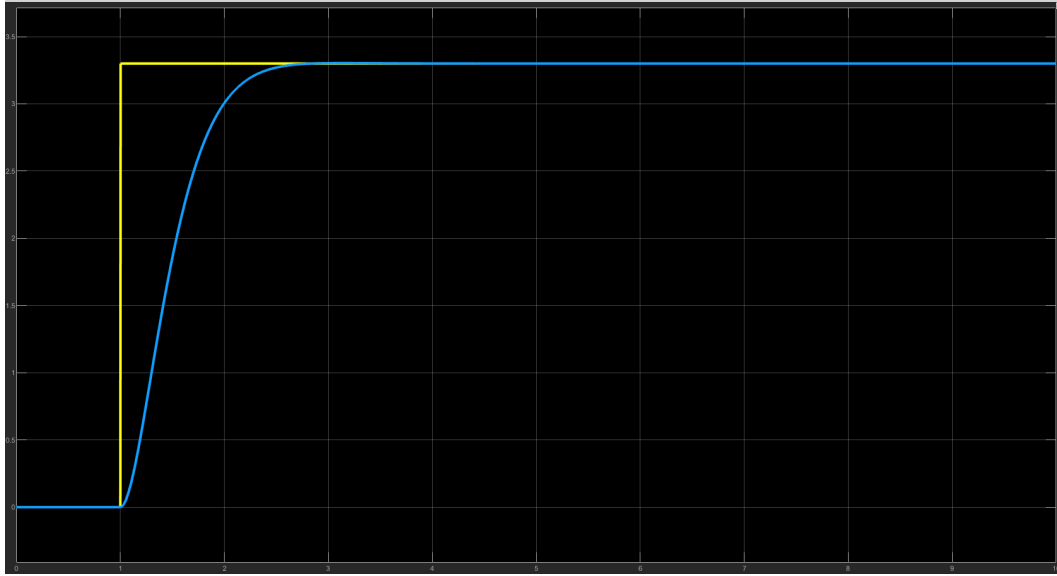


FIGURE 4.9 – Asservissement en position par tachymétrie en simulation

On choisit $K1 = 0.07$, $K2 = 1.2$ et $B = 0.005$. $K1$ permet d'ajuster la marge de phase, $K2$ l'écart statique et B permet d'amortir l'ensemble.

Sur 4.9, la réponse est rapide, sans dépassement visible, la stabilisation se fait en environ 1,5–1,6 s.

4.4.4 Retour tachymétrique sur système réel

Après implémentation sur le système réel (voir Figure 4.10) et amélioration des gains (voir Figure 4.10), la consigne de 180° est atteinte en 0,327 s voir 4.11, avec un supposé léger dépassement, ce qui est jugé très correct. Il y a en fait un écart initial : la position de départ réelle est légèrement différente de $\theta = 0$ (verticale basse exacte), donc le léger décalage en début de trajectoire se voit dans la consigne car le déplacement fait est de $+180^\circ$.

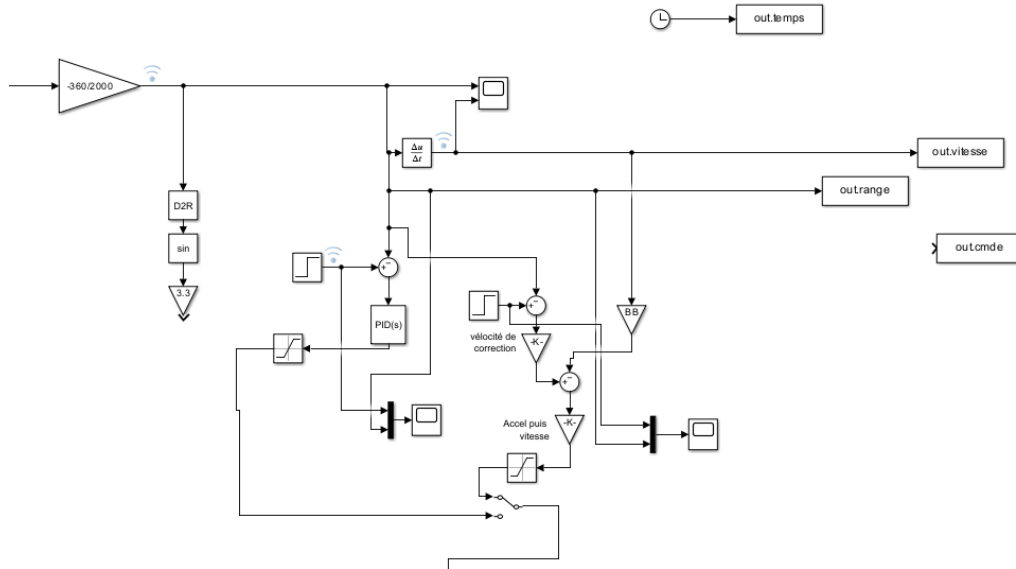


FIGURE 4.10 – Schema simulink de l'asservissement en position par tachymétrie en test physique



FIGURE 4.11 – Asservissement en position par tachymétrie en test physique

4.5 Test supplémentaire

Nous faisons varier la valeur d'échantillonnage des données réelles. La dérivée réelle est bruitée par l'échantillonnage, contrairement à la dérivée continue en simulation. L'effet testé d'une fréquence d'échantillonnage plus élevée est donc censé abimer le temps de réponse. La quantification du codeur (2000 incréments/tour en quadruple, soit $0,18^\circ$) est alors divisée par T_s dans la dérivée. Cependant, on a pu constater une légère amélioration de la vitesse de convergence.

5 Conclusion

Ce TP a permis de confronter les modèles théoriques de commande à la réalité physique du système. On a pu mettre en évidence la complexité liée aux non-linéarités.

Ainsi, l'approche MIL-SIL-HIL aide à concevoir une commande robuste et sécurisée. Le MIL a permis de poser les bases de modélisation de la dynamique fondamentale du moteur. L'étape SIL a été un premier environnement de simulation pour concevoir la commande linéarisante. Enfin, le HIL a permis de confronter la théorie à la réalité.

Lors des essais sur le système réel, plusieurs limites physiques et matérielles ont perturbé le modèle idéal, notamment la présence d'un frottement sec important et un décalage de calibration de la tension sur la carte d'amplification.

La mise en pratique a démontré les limites d'un simple correcteur PID en position face au couple de gravité non linéaire du balourd. À l'inverse, nous avons découvert l'asservissement par retour tachymétrique, qui a été très efficace sur le système réel.

6 Annexe

6.1 Tableau des paramètres

TABLE 6.1 – Paramètres du moteur AMAX32G et du pendule

Paramètre	Symbole	Valeur	Unité
Tension d'alimentation	U_a	12	V
Résistance aux bornes	R	2,99	Ω
Inductance	L	0,42	mH
Constante de couple	K_c	24,1	mN·m/A
Constante de vitesse	K_e	$24,1 \times 10^{-3}$	V·s/rad
Inertie rotor seule	J_{mot}	$41,4 \times 10^{-7}$	kg·m ²
Frottement visqueux (moteur seul)	b	$7,2 \times 10^{-5}$	N·m/(rad/s)
Inertie équivalente (moteur seul)	J_{eq}	$6,87 \times 10^{-5}$	kg·m ²
Masse du balourd	M	29	g
Longueur du pendule	L_p	80	mm
Couple de basculement	C_r	0,266	N·m
Tension de basculement	U_{basc}	3,3	V
Inertie équivalente (avec balourd)	J_{eq}^{bal}	$2,54 \times 10^{-4}$	kg·m ²

6.2 Codes Matlab

```

1 Ke=1/396 *60/(2*pi); % V/rad/s =0.241 =Kc
2 L=0.42*10^(-3); %Henri
3 J=41.4 * 10^(-7); %gcm^2 -> kgm^2
4 Kc=24.1 * 10^(-3); %Nm/A
5 Cr=0;
6 Fs=0;
7 %b=1/(49.1*10^(3)*2*pi/60); %tr/min/mNm -> N.m/(rad/s)
8 R=2.99; %Ohm
9
10
11 M=J/L^2;
12 g=9.81;
13
14 %a vide
15 I_vide = 85.9 * 10^(-3); % A
16 Omega_vide = 4630 * 2 * pi / 60; % rad/s
17 b = (Kc * I_vide) / Omega_vide; % Donne environ 4.27e-6 N.m/(rad/s)
18
19
20
21 motor = ss(motor1.A, motor1.B, motor1.C, motor1.D);
22 tf(motor)
23 zpk(motor)

```

```
24 [z1, p1, k1] = zpkdata(motor);
```

Listing 6.1 – TP1 : MIL et SIL

```
1 R=2.99;  
2 L=0.42e-3;  
3  
4 Ke=24.1e-3;  
5 Kc=Ke;  
6 J_m = 41.4e-7;  
7 J_c = 0.60e-7;  
8 %J=J_c+J_m;  
9  
10 I_vide=85.9e-3;  
11 omega_vide=4630*(2*pi)/60;  
12 %b_m=0;  
13  
14 %exo N.m/rad/s  
15 b=7.2e-5;  
16 %exo  
17 J=6.87e-5;  
18 %J balourd  
19 Jbal=3.19e-3;  
20  
21  
22 K_amp=2.34;  
23 Te=0.001;  
24  
25 l=0.08;  
26 g=9.81;  
27 m=0.029;  
28  
29 C_balourd=m*g*l;  
30 I_max=1.580;  
31  
32  
33 u=3.3;  
34 C_r=u*Kc/R;  
35  
36 Jeq=0.029*l^2 + J;  
37  
38 K1=0.007;  
39 K2=1;  
40 B=0.003;  
41  
42 KK1=0.07;  
43 KK2=1.2;  
44 BB=0.005;  
45  
46 Kpose=1;
```

Listing 6.2 – TP2 : HIL